Reviewer Pack — Matroid Rank Lower Bound for Monotone k-CLIQUE

Entry cf5188cf0224 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 03:08:49 UTC

Matroid Rank Lower Bound for Monotone k-CLIQUE

Entry ID: `cf5188cf0224` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 03:08:49 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

> For a monotone DNF formula φ, let M_φ be the matroid whose ground set is clauses of φ and rank function r(S) = max{|S'| : S' ⊆ S, S' is a collection of pairwise disjoint clauses}. Then r(M_φ) = O(log n) for all φ with circuit size ≤ n^c, but r(M_φ) = Ω(n) for all k-CLIQUE instances with k ≥ 3.

Rationale (proposer's reasoning):

> Matroid rank captures combinatorial independence, which may mirror the structural constraints of monotone circuits. High rank for k-CLIQUE suggests inherent complexity, while low rank for general formulas hints at decomposability, offering a novel angle to circuit lower bounds.

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9ba87ebf63a35f2c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import random
from itertools import combinations

def run_trial(seed: int) -> dict:
    def generate_random_formula(n, c):
        clauses = []
        for _ in range(c):
            clause = [random.randint(1, n), random.randint(1, n), random.randint(1, n)]
            while len(set(clause)) != 3:
                clause = [random.randint(1, n), random.randint(1, n), random.randint(1, n)]
            clauses.append(tuple(sorted(clause)))
        return clauses

    def is_disjoint(S):
        for i in range(len(S)):
            for j in range(i + 1, len(S)):
                if set(S[i]) & set(S[j]):
                    return False
        return True

    def matroid_rank(clauses):
        rank = 0
        for k in range(1, len(clauses) + 1):
            for S in combinations(clauses, k):
                if is_disjoint(S):
                    rank = max(rank, k)
        return rank

    n = random.randint(5, 40)
    c = int(n ** 2.3)  # Ensure circuit size ≤ n^c
    formula = generate_random_formula(n, c)

    matroid_ranks = [matroid_rank(formula) for _ in range(50)]
    avg_rank = sum(matroid_ranks) / len(matroid_ranks)
    support_fraction = sum(1 for r in matroid_ranks if r <= 5 * math.log(n)) / len(matroid_ranks)

    return {
        "metric_name": "matroid_rank",
        "metric_value": avg_rank,
        "instances_tested": len(matroid_ranks),
```

```
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": "" if support_fraction >= 0.8 else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]
    results = []

    for seed in seeds:
        random.seed(seed)
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    avg_rank = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_rank} std=??? support_fraction={support_fraction}")
    else:
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={seeds[0]}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111487
2	propose	ollama_remote	qwen3:8b	0	0	49689
3	preregistration	ollama_remote	qwen3:8b	0	0	26334
4	novelty	ollama_remote	qwen3:8b	0	0	20722

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	qwen3:8b	0	0	17485
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11511
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6795
8	judge	ollama_remote	qwen3:8b	0	0	15257

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 259279 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/cf5188cf0224.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cf5188cf0224.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cf5188cf0224.tar.gz` (if generated)